SENIOR ARCHITECTURE NOTES

Svelte 5 Mastery

High-density technical reference for developers migrating to the Runes-based reactivity model. All examples use Svelte 5 syntax exclusively.

`svelte@5.x` `sveltekit@2.x` `runes mode` `no legacy compat` `typescript-ready`

TABLE OF CONTENTS

01 The Reactivity Engine (Runes)

02 Component Communication

03 Logic & Flow Control

04 Advanced State Management

05 SvelteKit Architecture

06 Laravel Developer Context

01 The Reactivity Engine (Runes)

Svelte 5 replaces the compiler-magic of Svelte 4 with an explicit, **signal-based reactivity primitive** called Runes. A rune is a function-like syntax that begins with `$` and is understood by the Svelte compiler — they're *not* runtime imports, they compile away entirely.

WHAT CHANGED FROM SVELTE 4

Svelte 4 inferred reactivity by scanning `let` declarations inside `<script>` and tracking assignments. This worked but was brittle with deeply-nested mutations, non-component JS files, and complex derived logic. Svelte 5 adopts **fine-grained signals** (similar to SolidJS), meaning reactivity is *explicit and universal*.

$state — Reactive State

The fundamental reactive primitive. Think of it as Laravel's `session()->put()` — a tracked container whose changes propagate automatically through the UI.

SVELTE

```
<script>
  // Scalar — direct value tracking
  let count = $state(0);

  // Object — deep reactive proxy (like Vue's reactive())
  let user = $state({ name: 'Ben', age: 22 });

  // Array — mutations like .push() are tracked
  let items = $state(['a', 'b']);

  function add() {
    count++;                  // ✅ tracked
    user.name = 'Benjamin';   // ✅ deep proxy — tracked
    items.push('c');          // ✅ tracked
  }
</script>

<button onclick={add}>{count}</button>
<p>{user.name}</p>
```

⚠ GOTCHA — OBJECT REASSIGNMENT

For objects and arrays, **mutations are tracked** (proxy-based). However, if you need to replace the entire object, reassign the variable: `user = { name:`

`'New' }`. Spreading into a new object and assigning also works: `user = { ...user, name: 'New' }`.

$state.raw — Escape Hatch

For large, non-reactive data structures (e.g. a canvas pixel buffer) where you want to avoid proxy overhead:

```svelte
let bigData = $state.raw({ pixels: new Uint8Array(1024) });
// Mutations are NOT tracked. Must fully reassign to trigger update.
bigData = { ...bigData, processed: true };
```

$derived — Computed Values

Pure, lazy computations that re-run only when their reactive dependencies change. Equivalent to a memoized computed property. **Never call $derived to produce side effects** — that's $effect's job.

```
<script>
  let price  = $state(100);
  let qty    = $state(3);

  // Simple expression
  let total  = $derived(price * qty);

  // Complex logic — use $derived.by()
  let summary = $derived.by(() => {
    const vat      = total * 0.15;
    const formatted = new Intl.NumberFormat('en-ZA', { style: 'currency', cu
    return `${formatted.format(total + vat)} incl. VAT`;
  });
</script>
```

SVELTE

```svelte
<p>{summary}</p>
```

$effect — Side Effects

Runs after the component mounts and re-runs whenever its reactive dependencies change. Use for DOM manipulation, logging, subscriptions, and syncing external state. Return a teardown function to clean up.

SVELTE

```svelte
<script>
  let query = $state('');

  $effect(() => {
    // Runs on mount + whenever `query` changes
    const timer = setTimeout(() => {
      console.log(`Searching: ${query}`);
    }, 300);

    // Teardown — cancels debounce before next re-run
    return () => clearTimeout(timer);
  });

  // $effect.pre runs BEFORE the DOM is updated
  $effect.pre(() => {
    // Useful for reading DOM measurements before a repaint
  });
</script>
```

⚠ GOTCHA — $EFFECT DEPENDENCY TRACKING

$effect tracks reactive values that are **read synchronously** inside it. If you read a $state value inside a callback (e.g. inside setTimeout), it won't be tracked. Read it before the callback and capture it in a local variable first.

★ PRO-TIPS — REACTIVITY ENGINE

→ Use `$derived` for anything "calculated from state" — never store computed values in $state and update them manually.

→ `$effect` should never modify the reactive state it reads — this creates infinite loops. Modify state in event handlers instead.

→ Use `$state.snapshot(value)` to get a plain non-reactive copy of a $state object for logging or sending over the network (`JSON.stringify` a proxy gives weird results).

→ Runes work in `.svelte.ts` files too — reactivity is not locked to components (see Section 04).

→ There is no "batching" API — Svelte 5 batches synchronous updates automatically in the same microtask.

02 Component Communication

Svelte 5 replaces `export let` with the `$props()` rune. Props are now destructured from a single function call, making TypeScript inference trivial and the contract explicit.

$props — Declaring Props

```
<script lang="ts">
  // All props destructured from a single $props() call
  let {
    name,
    age = 18,          // default value
    class: cls = '',   // renaming (class is reserved)
    ...rest            // spread remaining attrs onto element
  }: {
    name: string;
```

```svelte
    age?: number;
    class?: string;
  } = $props();
</script>

<div class={cls} {...rest}>
  {name}, {age}
</div>
```

SVELTE 4 (LEGACY)	SVELTE 5 (RUNES)
`export let count = 0;`	`let { count = 0 } = $props();`
`export let class = '';` (syntax error)	`let { class: cls } = $props();`
`$$props / $$restProps`	`let { known, ...rest } = $props();`

THE PROXY SYSTEM UNDER THE HOOD

In Svelte 4, `export let` used the compiler to wire up a setter. In Svelte 5, the parent passes a **reactive proxy** — when the parent's state changes, the proxy notifies the child automatically. This means prop changes are *fine-grained*: only components that read a changed prop re-render, not the entire subtree.

$bindable — Two-Way Binding

Opt-in two-way binding. The child *declares* a prop as bindable; the parent *opts in* with `bind:`. This makes the data flow contract explicit at the API surface.

```svelte
// Child: Counter.svelte
<script>
  let { value = $bindable(0) } = $props();
</script>

<button onclick={() => value++}>+</button>
```

```svelte
<span>{value}</span>
<button onclick={() => value--}>-</button>
```

SVELTE (CHILD)

```svelte
// Parent: App.svelte
<script>
  import Counter from './Counter.svelte';
  let qty = $state(1);
</script>

<Counter bind:value={qty} />
<p>Qty in parent: {qty}</p>
```

SVELTE (PARENT)

Snippets — Typed Slot Replacement

Snippets replace the slot system. They're **typed, composable fragments** that can be passed as props or defined locally. Think of them as renderProps in React, but with zero boilerplate.

```svelte
// Parent passes a named snippet as a prop
<DataTable data={rows}>
  {#snippet header()}
    <th>Name</th><th>Email</th>
  {/snippet}

  {#snippet row(item)}
    <td>{item.name}</td><td>{item.email}</td>
  {/snippet}
</DataTable>

// DataTable.svelte — receives and renders snippets
<script>
  import type { Snippet } from 'svelte';
  let { data, header, row }: {
```

```svelte
    data: any[];
    header: Snippet;
    row: Snippet<[any]>;
  } = $props();
</script>

<table>
  <thead><tr>{@render header()}</tr></thead>
  <tbody>
    {#each data as item}
      <tr>{@render row(item)}</tr>
    {/each}
  </tbody>
</table>
```

★ PRO-TIPS — COMPONENT COMMUNICATION

→ Props in $props() destructuring are **live references** to the parent's reactive state. Do NOT reassign destructured props directly unless using $bindable — the parent won't know.

→ Use `{@render children?.()}` (with optional chaining) when `children` is an optional default snippet — avoids runtime errors if no children are passed.

→ For component libraries, document which props are `$bindable` explicitly — it's part of your API contract now.

→ Snippets are first-class values — you can store them in $state, pass them through multiple layers, and render them conditionally. This unlocks advanced patterns like plugin systems.

03

Logic & Flow Control

Svelte uses template directives compiled to optimized imperative DOM operations — no virtual DOM diffing. The result is the smallest and fastest update path possible.

{#if} — Conditional Rendering

SVELTE

```svelte
{#if user.role === 'admin'}
  <AdminPanel />
{:else if user.role === 'editor'}
  <EditorDashboard />
{:else}
  <p>Access denied.</p>
{/if}
```

KEYED {#IF} VS CSS VISIBILITY

`{#if}` **destroys and recreates** the DOM node. Use it when you want component lifecycle to reset. Use `style:visibility` or `style:display` when you want to hide but preserve state (e.g. a form mid-fill). This is the same distinction as `v-if` vs `v-show` in Vue.

{#each} — List Rendering

```svelte
<!-- Basic -->
{#each items as item}
  <li>{item.name}</li>
{/each}

<!-- With index -->
{#each items as item, i (item.id)}
```

```svelte
  <li>{i + 1}. {item.name}</li>
{/each}

<!-- Keyed + empty state -->
{#each items as item (item.id)}
  <ProductCard {item} />
{:else}
  <p>No items found.</p>
{/each}
```

⚠ GOTCHA — ALWAYS KEY DYNAMIC LISTS

Without a key expression `(item.id)`, Svelte reuses DOM nodes by position. When items are reordered or removed, components keep their old state — causing subtle bugs. Always add a unique key when the list can change dynamically.

{#await} — Async / Promise Handling

Handles the loading/resolved/rejected states of a Promise declaratively. The cleanest async pattern in any UI framework.

```svelte
<script>
  import { fetchUser } from '$lib/api';
  let userId = $state(1);
</script>

<!-- Re-fetches automatically when userId changes -->
{#await fetchUser(userId)}
  <Spinner />
{:then user}
  <h2>{user.name}</h2>
{:catch error}
  <p class="error">{error.message}</p>
{/await}
```

SVELTE

```svelte
<!-- Short form — no loading state needed -->
{#await fetchUser(userId) then user}
  <p>{user.email}</p>
{/await}
```

{@render} — Rendering Snippets

SVELTE

```svelte
<script>
  import type { Snippet } from 'svelte';
  let { header, children } = $props();
</script>

<!-- Render a named snippet -->
{@render header()}

<!-- Render default children (replaces <slot />) -->
{@render children()}

<!-- Conditional rendering of optional snippet -->
{#if header}{@render header()}{/if}
```

{@html} and {@debug}

SVELTE

```svelte
<!-- Raw HTML — sanitize first! XSS risk -->
{@html sanitizedMarkdown}

<!-- Pauses dev tools when values change -->
{@debug user, count}
```

★ PRO-TIPS — LOGIC & FLOW

→ In SvelteKit, prefer server-side data loading over `{#await}` for initial page data — `{#await}` is best for *user-triggered* async actions (search, load more, etc.).

→ `{#each}` works on any iterable — Sets, Maps, generator functions. You're not limited to arrays.

→ Snippets defined inside an `{#each}` block close over the loop variable — useful for dynamic render functions without writing a child component.

→ `{@const x = expr}` lets you declare a local variable inside a template block, keeping derived display logic out of `<script>`.

04 Advanced State Management

Svelte 5 eliminates the need for `writable` / `readable` stores from `svelte/store`. The new pattern is **universal reactive classes and functions** using the `.svelte.ts` file extension, where all Runes work identically to inside components.

The .svelte.ts Pattern

Any TypeScript file with the `.svelte.ts` extension can use Runes. This is the foundation of all shared state in Svelte 5.

```
// src/lib/cart.svelte.ts
import type { Product } from './types';

function createCart() {
  let items = $state<Product[]>([]);
  let total = $derived(items.reduce((sum, p) => sum + p.price, 0));

  return {
```

```ts
    get items() { return items },   // expose reactively (getter)
    get total() { return total },   // derived is also exposed via getter
    add(product: Product) { items.push(product) },
    remove(id: string)   { items = items.filter(p => p.id !== id) },
    clear()              { items = [] }
  };
}

// Module-level singleton — shared across ALL components
export const cart = createCart();
```

```svelte
// Any component — just import and use
<script>
  import { cart } from '$lib/cart.svelte';
</script>

<p>Total: R{cart.total.toFixed(2)}</p>
<button onclick={() => cart.clear()}>Clear</button>
```

> ⚠ **GOTCHA — THE GETTER PATTERN IS CRITICAL**
>
> If you return `{ items, total }` instead of `{ get items() { return items } }`, you capture the *initial value* as a plain variable — reactivity breaks. The getter pattern ensures the property lookup always returns the current reactive value.

Class-Based State (The OOP Approach)

For complex domains (auth, forms, websocket connections), a class encapsulates logic cleanly:

```ts
// src/lib/auth.svelte.ts
export class AuthStore {
  user  = $state<User | null>(null);
```

```ts
  token = $state<string | null>(null);

  get isAuthenticated() { return this.user !== null; }

  async login(email: string, password: string) {
    const res = await fetch('/api/login', { ... });
    const { user, token } = await res.json();
    this.user  = user;
    this.token = token;
  }

  logout() {
    this.user  = null;
    this.token = null;
  }
}

export const auth = new AuthStore();
```

Context API — Scoped Tree State

When you want shared state scoped to a subtree (not global), combine Svelte's Context API with $state:

```
// Parent component — sets context
<script>
  import { setContext } from 'svelte';

  const theme = $state({ color: 'dark', font: 'mono' });
  setContext('theme', {
    get color() { return theme.color },  // reactive getter
    setColor: (c: string) => theme.color = c
  });
</script>
```

```svelte
// Deep child — consumes without prop drilling
<script>
  import { getContext } from 'svelte';
  const theme = getContext('theme');
</script>

<div class={theme.color}>Themed content</div>
```

SVELTE

GLOBAL MODULE VS CONTEXT VS SVELTEKIT LOAD()

Module singleton (.svelte.ts): Use for truly global state (cart, auth, theme preferences). Shared across all pages, persists during navigation.

Context API: Use for state shared within a subtree (a data table's sort/filter state, a form wizard's step state). Scoped and garbage-collected with the tree.

SvelteKit load() data: Use for page-specific server data. Flows down via $page.data. Not persistent — re-runs on navigation.

★ PRO-TIPS — ADVANCED STATE

- → Module singletons are **per-server-request in SSR** — if you're using SSR and state is truly per-user, initialize it in SvelteKit's `load()` and pass it via context, not a module singleton.
- → You can use `$effect` inside a `.svelte.ts` function, but only inside Svelte's component tree. If the function is called outside a component, effects won't run. Use an explicit setup/teardown pattern instead.
- → Name your state files `*.svelte.ts` — the extension is what enables rune support. A plain `*.ts` file will throw compiler errors if you try to use Runes.

05

SvelteKit Architecture

SvelteKit is the opinionated, full-stack framework built on Svelte — it handles routing, server-side rendering, data loading, and form actions. It's the equivalent of Laravel for the Svelte ecosystem, though the architecture is fundamentally different.

Filesystem Router — File Conventions

FILESYSTEM

```
src/routes/
├── +layout.svelte        ← Persistent shell (nav, footer)
├── +layout.server.ts     ← Layout-level auth guard load()
├── +page.svelte          ← / (home)
├── +page.server.ts       ← Server-only: load() + actions
├── +page.ts              ← Universal: load() runs server + client
├── +error.svelte         ← Error boundary UI
│
├── products/
│   ├── +page.svelte      ← /products
│   └── [id]/
│       ├── +page.svelte  ← /products/[id]
│       └── +page.server.ts
│
└── api/
    └── webhook/
        └── +server.ts    ← REST endpoint (GET/POST/etc.)
```

load() — Server Data Fetching

The `load()` function in `+page.server.ts` runs *exclusively on the server*. This is where you query your database, validate sessions, and fetch secrets. Its return value is merged into `$page.data` and typed automatically.

TS

```ts
// src/routes/products/[id]/+page.server.ts
import type { PageServerLoad } from './$types';
import { error } from '@sveltejs/kit';
import { db } from '$lib/db';

export const load: PageServerLoad = async ({ params, locals }) => {
  // locals — set by hooks.server.ts (e.g., session user)
  if (!locals.user) error(401, 'Unauthorized');

  const product = await db.product.findUnique({
    where: { id: params.id }
  });

  if (!product) error(404, 'Product not found');

  return { product };  // serialised and sent to the client
};
```

SVELTE

```svelte
// +page.svelte — typed data, no await, no boilerplate
<script>
  import type { PageData } from './$types';
  let { data }: { data: PageData } = $props();
</script>

<h1>{data.product.name}</h1>
<p>R{data.product.price}</p>
```

actions — Form Mutations

Actions handle form `POST` mutations on the server. Use `enhance` for progressive enhancement — the form works without JS, and with JS it becomes a fetch-based SPA action.

TS

```ts
// +page.server.ts — named actions
import type { Actions } from './$types';
import { fail, redirect } from '@sveltejs/kit';

export const actions: Actions = {
  async createProduct({ request, locals }) {
    const data = await request.formData();
    const name  = data.get('name') as string;
    const price = Number(data.get('price'));

    if (!name) return fail(400, { error: 'Name required', name });

    await db.product.create({ data: { name, price } });
    redirect(303, '/products');
  }
};
```

SVELTE

```svelte
// +page.svelte — progressive enhancement
<script>
  import { enhance } from '$app/forms';
  let { form } = $props(); // contains action return values
</script>

<form method="POST" action="?/createProduct" use:enhance>
  <input name="name" value={form?.name ?? ''} />
  {#if form?.error}
    <p class="error">{form.error}</p>
  {/if}
  <button>Create</button>
</form>
```

hooks.server.ts — Middleware

```ts
// src/hooks.server.ts — runs on every request
import type { Handle } from '@sveltejs/kit';

export const handle: Handle = async ({ event, resolve }) => {
  // Auth — populate locals before any load() runs
  const token = event.cookies.get('session');
  event.locals.user = token ? await validateToken(token) : null;

  return resolve(event);
};
```

★ PRO-TIPS — SVELTEKIT

→ Use `+page.server.ts` (not `+page.ts`) whenever you access databases, secrets, or cookies — it **never ships to the client bundle**.

→ `fail()` preserves form input and validation messages without a redirect — this mirrors Laravel's `back()->withErrors()->withInput()` pattern exactly.

→ SvelteKit auto-generates `./$types` — `PageData`, `Actions`, `PageServerLoad`. Always import from here for full end-to-end type safety.

→ Use `invalidate('tag:products')` or `invalidateAll()` from `$app/navigation` to force a re-run of `load()` after a client-side mutation (e.g., after a fetch-based delete).

06

Laravel Developer Context

You already think in controllers, requests, and views. Here's a direct conceptual mapping to SvelteKit so you're not learning from scratch — you're re-mapping.

LARAVEL CONCEPT	SVELTEKIT EQUIVALENT	KEY DIFFERENCE
Route (routes/web.php)	Filesystem route (+page.svelte)	Convention over configuration — file path IS the URL
Controller + Request	+page.server.ts load()	No separate class — load() is the controller method
POST Route + Form Request	+page.server.ts actions	Colocated with the page — no separate controller needed
Middleware (Kernel.php)	hooks.server.ts handle()	Single intercept function, not a class stack
Blade Template	+page.svelte (template section)	Client-side reactive — not server-rendered string concatenation
Service Provider	No direct equivalent	Use module singletons (.svelte.ts) for shared services
session()->flash()	fail() / return from action	Returns serialized data to the page, no server session needed
Eloquent Model	Prisma / Drizzle / Kysley	No official ORM — ecosystem choice (Prisma is most common)

The Fundamental Mental Model Shift

LARAVEL — REQUEST/RESPONSE LIFECYCLE

Every user interaction (click a link, submit a form) triggers a **full HTTP round-trip**. PHP re-boots, runs the route, queries the DB, renders a Blade template to a

string, and returns the full HTML. State lives in the database and session. The browser replaces the page entirely.

SVELTEKIT — HYBRID LIFECYCLE

First load: Server runs `load()`, serializes data to JSON, renders HTML (SSR), hydrates in the browser — nearly identical to Laravel for the initial hit.

Navigation: SvelteKit intercepts link clicks, fetches only the JSON data from the new page's `load()` function, and surgically updates just the changed parts of the DOM. *No full page reload.* State lives in Svelte's reactive system — in-memory, in the browser.

Form submission with enhance: Fetch replaces the POST. Server runs the action, returns data. Browser updates the UI fragment. No reload, no scroll-to-top, no flash of blank content.

The PRG Pattern — You Already Know This

SvelteKit's `actions` + `redirect(303, ...)` is the same **POST-Redirect-GET** pattern you use in Laravel. The `fail()` function mirrors `back()->withErrors()->withInput()`. With `use:enhance`, this happens over fetch without reloading the page, but the mental model is identical — the server is still the source of truth for mutations.

```
// Laravel mental model → SvelteKit translation

// LARAVEL:
// Route::post('/products', [ProductController::class, 'store'])
// return back()->withErrors(['name' => 'Required'])->withInput();
// return redirect('/products')->with('success', 'Created!');

// SVELTEKIT (+page.server.ts actions):
export const actions = {
  async store({ request }) {
    const fd = await request.formData();
    if (!fd.get('name')) {
      return fail(422, { errors: { name: 'Required' }, input: Object.fromEntries(fd)
```

```ts
    }
    await createProduct(fd);
    redirect(303, '/products');
  }
};
```

★ PRO-TIPS — LARAVEL MIGRATION CONTEXT

→ Svelte's `{#if}`, `{#each}` are functionally equivalent to Blade's `@if`, `@foreach` — but they react to state changes in real-time without a server round-trip.

→ Filament (your stack) and SvelteKit are actually complementary — use Filament for your admin dashboard (where Laravel shines), and SvelteKit for the public-facing client app (where reactivity and UX shine).

→ Laravel's `Event` + `Listener` pattern maps loosely to Svelte's `$effect` watchers — both react to state changes and trigger side-effects.

→ In Svelte 5, `onclick` (no colon) replaces `on:click` — it's now standard HTML event attributes. Easier to remember: write HTML attributes like you normally would.

→ SvelteKit's `$app/navigation` functions (`goto()`, `preloadData()`) are the client-side equivalent of Laravel's `redirect()` — but they run without leaving the page.

Svelte 5 Mastery Notes — Benjamin @ Stratusolve

All examples: Svelte 5 / SvelteKit 2 / Runes mode